

3교시 · RAG 2 — 검색은 왜 실패하고 어떻게 고치는가

2교시 저장소를 그대로 사용. 이 교시의 진짜 주제는 개별 기법이 아니라 "고치는 순서" — 어떤 시스템에도 통하는 절차다.

고치는 순서 (4단계, 범용 기술)

증상 확인 → 원인 분리 → 처방 선택 → 결과 대조(숫자로)

증상 ① — 근거는 둘인데 답은 하나다

2교시 마지막 질문(**AEC-Q100 Grade 1의 HTOL 시험은 몇 시간?**)의 답변과 근거 청크를 원문과 대조 — 답변에 없는 숫자가 근거 안에 있는지 확인.

원인 후보 두 가지

- 생성이 잘못했다 (근거는 맞는데 AI가 엉뚱하게 요약)
- 검색이 잘못했다 (애초에 넘어간 근거 자체가 문제 — 예: 서로 다른 판(Rev)의 값이 섞여 있음)

판별법: 근거 청크를 직접 읽는다. 답변 내용이 근거 안에 실제로 있으면 생성은 제 역할을 한 것 → 문제는 검색(그 앞 단계)에 있음.

처방 1 — Reranking (재정렬)

- 임베딩 검색은 질문과 조각을 **각각 따로** 숫자로 바꿔 거리만 잰다 (조각 벡터는 미리 계산 — 빠름). 조각 벡터를 만들 때 질문은 존재하지 않았으므로 질문의 세부 조건이 반영될 자리가 없음.
- **Reranking** = 질문과 조각을 **함께 읽고** 다시 순위를 매기는 단계. 조각마다 모델을 한 번씩 불러야 해서 느리고 비쌌 → **2단계로** 나눔:
1차 검색 전체 청크 → 후보 10개 싸다·빠르다 2차 재정렬 후보 10개 → 최종 5개 비싸다·느리다
 1차는 "정답을 후보 안에 넣는 일", 2차는 "후보 중 진짜를 위로 올리는 일". **1차에서 빠지면 2차가 되살릴 수 없음** → Reranking 붙일 땐 1차 K를 오히려 늘림.
- 오늘 쓰는 OpenAI API에는 전용 reranker가 없어 LLM(**gpt-5.6-luna**)에게 후보+질문을 주고 순서만 매기게 함 = **LLM-as-a-reranker** (전용 모델보다 느리지만 바로 붙일 수 있음).

개발 요청:

검색에 **Reranking** 을 추가해 줘.

- 재정렬을 켜고 끌 수 있는 토글을 만들어 줘.
- 재정렬을 활성화한 경우에는 하이브리드 검색만 수행하도록 설정해줘.
- 1차 검색으로 후보 10개를 꺼내고, 그 후보를 질문과 함께 LLM 에 넘겨 다시 순서를 매겨 줘.
- 재정렬 결과 상위 5개를 답변 생성의 근거로 써 줘.
- 모델은 **gpt-5.6-luna** 를 쓰고, 재정렬에 걸린 시간을 화면에 표시해 줘.
- 변경된 상황에 맞게 결과를 표시하는 UI를 깔끔하게 정리해줘.

(→ 앱 재시작 필요. 이번 교시 개발 요청 4번 모두 동일)

재정렬이 모르는 것: 재정렬은 "이 조각이 질문에 얼마나 답이 되는가"만 봄 — 조각이 어느 문서/몇 쪽/어떤 기준에서 나왔는지는 모름.

처방 2 — Metadata Filter

저장소의 각 청크에는 원문·벡터 외에 **문서명/쪽/청킹 기준** 같은 메타데이터가 함께 저장돼 있음 → 이를 검색 조건으로 써서 **찾을 범위 자체를 좁히는 것** = Metadata Filter.

검색에 문서 단위 필터를 추가해 줘.

- 저장소에 적재된 문서 목록을 화면에 보여주고, 체크박스로 검색 대상을 고를 수 있게 해 줘.
- 고른 문서 안에서만 검색이 일어나게 해 줘.
- 아무것도 고르지 않으면 전체를 대상으로 해 줘.

테스트: 필터 없음 / Rev H만 / Rev G만 → 세 조건으로 같은 질문을 던져 원본 PDF와 대조.

처방 선택의 원칙 — "증상에 맞는 처방"

- Reranking이 고치는 증상: 질문 맥락을 반영한 순위 재조정 (근거 후보 중 진짜를 위로)
- Metadata Filter가 고치는 증상: 애초에 섞이면 안 되는 범위(다른 판/문서)를 원천 차단
- 맞지 않는 처방은 느려지기만 하거나 더 나빠질 수 있음.

증상 ② — 짧은 질의가 헛돈다

2교시에서 BM25+하이브리드 이후에도 못 찾던 짧은 질의들(예: 6.2.1.1, 박리)이 있었다.

원인: 숫자뿐인 질의는 임베딩이 잡을 뜻이 거의 없음. 한 단어 질의(박리)는 저장소 밖 문맥에서 전혀 다른 뜻을 가질 수 있음. 사람은 맥락을 알고 물었지만 그 맥락이 질의 자체엔 없음.

처방 3 — Query Rewrite (질의 재작성)

검색 전에 질의를 검색에 맞게 다시 쓰는 단계.

1단계(맥락 없이):

검색 전에 질의를 다시 쓰는 단계를 추가해 줘.

- 사용자가 넣은 질의를 LLM에 넘겨 검색에 맞는 형태로 바꾸고, 바뀐 질의로 검색해 줘.
- 모델은 gpt-5.6-luna를 써 줘.
- 원래 질의와 다시 쓴 질의를 화면에 나란히 보여 줘.
- 이 단계를 켜고 끌 수 있는 토글을 만들어 줘.

2단계(고치기 — 저장소 맥락 부여):

질의 재작성 프롬프트를 고쳐 줘.

- 저장소에 어떤 문서들이 적재되어 있는지 목록을 프롬프트에 넣어 줘.
- 그 문서들의 맥락 안에서 질의를 해석하도록 지시해 줘.
- 저장소 밖의 뜻으로 넓히지 말라고 명시해 줘.
- 문서 제목이나 문서 목록을 다시 쓴 질의에 그대로 넣지 말라고 명시해 줘.

나아졌는지 확인하기 — 자동 채점 (LLM-as-a-Judge)

질문·기대답을 미리 정해두고 AI가 채점 → 정성적 판단을 정량화.

답변 품질을 채점하는 기능을 추가해 줘.

- 질문과 기대하는 답을 여러 개 적어 둔 파일을 읽어서, 각 질문을 앱에 던지고 답변을 받아 줘.
- 그 답변을 기대하는 답과 비교해 1~5점으로 채점하고 이유를 한 줄로 적게 해 줘.
- 채점도 gpt-5.6-luna 로 해 줘.
- 결과를 표로 보여주고 평균 점수를 계산해 줘.

공통 5문항(`grading.json` 형식) + 각자 2문항 추가. 조건별(무처방/필터만/재작성만/재정렬만/셋다)로 **평균 점수와 걸린 시간**을 표로 기록.

채점자도 틀린다 — AI가 매긴 점수를 그대로 믿지 말고, 이상한 점수 2~3개를 골라 사람이 원문으로 재 확인.

정리

- 기법을 아는 것과 어느 증상에 쓰는지 아는 것은 다르다
- 맞지 않는 처방은 느려지기만 하거나 더 나빠진다
- 재지 않으면 고친 것인지 망친 것인지 알 수 없다

지금 가진 것: `rag-lab` — naive RAG + BM25·하이브리드·Metadata Filter·Query Rewrite·Reranking·자동 채점

더 해보기

- Reranking 쿼리 채로 1차 후보 K를 10/20/50으로 바꿔보기
- 재정렬을 같은 질문에 2~3번 걸어 순서·답이 매번 같은지 확인
- 채점 프롬프트를 자세히 적으면 점수가 안정되는지 확인
- 세 처방을 켜는 순서를 바꿔보고 결과가 달라지는지 확인

다음 질문: 지금까지는 우리가 조건을 켜고 껐다. AI가 스스로 판단해서 도구를 고르게 하려면? → 4교시 Tool Calling